

DETECÇÃO VISUAL DE ANOMALIAS COM APOIO DA IA

Guia Prático do Aluno

Aplicação CopaFigurinhas | Módulo 2 | Dia 2

Propósito do laboratório

Modificar o código de forma controlada, executar cenários reais, visualizar os desvios em um painel e utilizar a IA para explicar as evidências sem permitir que ela substitua a decisão determinística do detector.

Material de estudo e prática orientada

1. Apresentação do laboratório

No primeiro dia, o foco esteve na explicação automática de falhas registradas durante lint, testes, build, integração contínua e deployment. Neste segundo dia, a pergunta muda. A aplicação pode continuar respondendo e, mesmo assim, apresentar um comportamento diferente do normal. A atividade foi construída para tornar esse desvio visível, mensurável e explicável.

Ao longo do material, você criará uma infraestrutura pequena, executará quatro cenários de anomalia e observará os resultados em um painel HTML. Em cada cenário, a análise deverá separar fatos medidos, regras acionadas, interpretação, hipótese e causa confirmada. Essa separação reduz conclusões precipitadas e mostra onde a IA realmente agrega valor.

Condição inicial

Este guia pressupõe que o repositório já foi restaurado para a branch main, validado e que uma nova branch de trabalho foi criada pelo professor. Os comandos para descartar as tentativas anteriores não fazem parte do material do aluno.

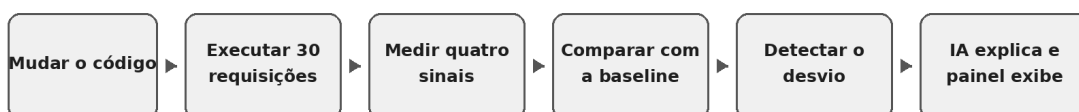
1.1 O que você deverá aprender

O objetivo não é decorar comandos. O objetivo é compreender um processo que pode ser reproduzido em outras aplicações. Ao final, você deverá conseguir explicar e aplicar os conceitos abaixo.

- distinguir uma falha explícita de uma anomalia de comportamento;
- definir um sinal observável e coletar amostras;
- criar uma baseline que represente uma condição normal;
- comparar valores observados com valores de referência;
- aplicar limites relativos e absolutos para reduzir falsos positivos;
- interpretar latência p95, taxa de erro, volume de logs e tamanho de resposta;
- usar a IA para explicar evidências sem entregar a ela a decisão do detector;
- confirmar a correção por meio de uma nova medição.

1.2 Fluxo central

Fluxo central do laboratório



A decisão de anomalia é determinística. A IA recebe os fatos depois da detecção.

Figura 1. Fluxo central do laboratório de detecção visual de anomalias.

A decisão de anomalia será produzida por regras transparentes. A IA entra depois da decisão para organizar evidências, interpretar o desvio, sugerir hipóteses e recomendar ações. Se a chave da OpenAI não estiver disponível, o laboratório continuará funcionando com uma explicação determinística.

1.3 Os quatro sinais observados

Sinal	Métrica	Pergunta respondida	Cenário
Tempo de resposta	latencyP95Ms	Parte relevante das respostas ficou lenta?	Latência intermitente
Confiabilidade	errorRate	Quantas requisições falharam na janela?	Erro intermitente
Volume operacional	logLinesPerRequest	A aplicação passou a gerar logs em excesso?	Excesso de logs
Eficiência da API	responseSizeP95Bytes	A resposta ficou muito maior do que o normal?	Payload inflado

Esses sinais são suficientes para demonstrar quatro tipos de desvio sem transformar o laboratório em uma plataforma complexa de observabilidade. O painel local é a interface visual do exercício. Ferramentas externas de tracing e alertas não fazem parte desta prática.

1.4 Como será o painel



Figura 2. Representação do painel visual esperado para um cenário de latência.

O painel apresentará quatro cartões, uma faixa visual com as requisições, evidências dos logs e a análise da IA. Os valores mostrados na figura são ilustrativos. Os números reais variarão de acordo com a máquina e com as condições da execução.

2. Preparação do ambiente de trabalho

Antes de solicitar qualquer alteração ao Claude Code, confirme que você está no branch de trabalho preparada para o Dia 2. Não execute os prompts diretamente na main.

Confirme a branch e o estado do repositório:

```
git branch --show-current  
git status --short
```

Instale as dependências e confirme que a base do Dia 1 permanece íntegra. Essa validação é importante porque o laboratório novo deve acrescentar funcionalidades sem quebrar os recursos anteriores.

```
npm ci  
npm run lint  
npm run test  
npm run build  
npm run ci
```

2.1 Ativação segura da explicação por IA

A chave da OpenAI nunca deve ser escrita no código, em um prompt, em uma captura de tela ou em um commit. Quando for necessário executar uma análise real, carregue a chave apenas no terminal que executará o cenário.

```
read -s -p "OPENAI_API_KEY: " OPENAI_API_KEY  
echo  
export OPENAI_API_KEY
```

Confirme apenas que a variável foi carregada. Não exiba a chave completa.

```
if [ -n "$OPENAI_API_KEY" ]; then  
  printf 'OPENAI_API_KEY carregada: %.5s...%s\n' "$OPENAI_API_KEY" "${OPENAI_API_KEY: -4}"  
else  
  echo "OPENAI_API_KEY está vazia."  
fi
```

Execução sem chave

Sem OPENAI_API_KEY, o painel continuará sendo gerado. A diferença será a origem da explicação, que aparecerá como fallback determinístico. O resultado da detecção deverá ser o mesmo com ou sem IA.

3. Construção do painel visual e do detector

A primeira entrega cria a infraestrutura comum. Neste momento, ainda não serão injetadas anomalias no backend. O objetivo é produzir a baseline, executar a aplicação normal, detectar desvios por regras simples e gerar o painel visual.

3.1 Por que cada arquivo será criado

Arquivo	Responsabilidade
automation/src/day2/observe.mjs	Executar requisições, capturar logs e transformar amostras em métricas.
automation/src/day2/detect.mjs	Comparar baseline e observação por regras determinísticas.
automation/src/day2/explain.mjs	Pedir à IA uma explicação da decisão já calculada.
automation/src/day2/report.mjs	Gerar JSON, Markdown e o painel HTML.
automation/src/day2/run.mjs	Orquestrar início do backend, coleta, detecção, explicação, relatório e encerramento.
automation/prompts/day-2-anomaly-explanation.md	Definir o contrato da explicação produzida pela IA.

A separação de responsabilidades permite que cada parte seja entendida e testada isoladamente. O observador mede, o detector decide, o explicador interpreta e o relatório apresenta. Misturar essas funções em um único arquivo dificultaria a manutenção e esconderia o processo de aprendizagem.

3.2 Prompt inicial para o Claude Code

Cole o prompt abaixo no Claude Code. Aguarde a implementação e leia o resumo apresentado antes de executar os comandos. Não aceite alterações que adicionem ferramentas ou métricas fora do escopo.

Você é um engenheiro de software sênior especializado em Node.js, Express, JavaScript ESM, logs estruturados, observabilidade e integração com OpenAI.

Trabalhe no repositório atual:

IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo

Branch:

feature/day-2-visual-anomaly-lab

CONTEXTO

O projeto já possui:

- aplicação CopaFigurinhas;
- backend Express;
- frontend React/Vite;
- logs estruturados JSON;

- rota GET /api/report;
- infraestrutura OpenAI;
- fallback determinístico;
- sanitizor de secrets;
- testes;
- Docker e CI.

O Dia 1 trabalhou logs explicados automaticamente pela IA.

O Dia 2 trabalhará detecção visual de anomalias.

OBJETIVO

Criar um laboratório pequeno e didático que:

1. inicia o backend como processo filho;
2. executa requisições reais a GET /api/report;
3. mede quatro sinais;
4. cria uma baseline;
5. compara uma execução observada com a baseline;
6. detecta anomalias por regras transparentes;
7. pede à IA uma explicação;
8. gera um painel HTML visual;
9. encerra o backend automaticamente.

MÉTRICAS

Meça somente:

- latencyP95Ms;
- errorRate;
- logLinesPerRequest;
- responseSizeP95Bytes.

Pode registrar requestCount como contexto.

Não implemente outras métricas decisórias.

ARQUIVOS

Crie somente:

```
automation/src/day2/observe.mjs
automation/src/day2/detect.mjs
automation/src/day2/explain.mjs
automation/src/day2/report.mjs
automation/src/day2/run.mjs
automation/prompts/day-2-anomaly-explanation.md
```

```
automation/tests/day-2-observe.test.mjs
automation/tests/day-2-detect.test.mjs
automation/tests/day-2-run.test.mjs
```

Ainda não crie demo-anomalies.js.
Ainda não altere a rota /api/report.

OBSERVAÇÃO

Cada amostra deve conter:

```
{
  "sequence": 1,
  "statusCode": 200,
  "durationMs": 4.2,
  "responseSizeBytes": 3820,
  "logLineCount": 2,
  "requestId": "..."
}
```

O laboratório deve:

- iniciar o backend na porta 3102;
- aguardar /api/health;
- fazer cinco requisições de aquecimento;

- descartar o aquecimento;
- realizar 30 requisições medidas;
- capturar stdout e stderr;
- interpretar somente linhas JSON válidas;
- relacionar logs pelo requestId quando possível;
- encerrar o processo em finally.

P95

Use um método didático de posto mais próximo:

```
index = Math.ceil(values.length * 0.95) - 1
```

BASELINE

Crie o comando:

```
npm run day2:baseline
```

Saída:

```
reports/day-2/baseline.json
```

Formato:

```
{
  "sampleCount": 30,
  "latencyP95Ms": 8,
  "errorRate": 0,
  "logLinesPerRequest": 2,
  "responseSizeP95Bytes": 3820,
  "createdAt": "...",
  "commitSha": "..."
}
```

DETECTOR

Implemente as regras:

LATÊNCIA:

```
observed.latencyP95Ms >= baseline.latencyP95Ms * 3
```

E

```
observed.latencyP95Ms - baseline.latencyP95Ms >= 150
```

ERRO:

```
observed.errorRate >= 0.15
```

E

```
observed.errorRate - baseline.errorRate >= 0.10
```

LOGS:

```
observed.logLinesPerRequest >= baseline.logLinesPerRequest * 3
```

E

```
observed.logLinesPerRequest - baseline.logLinesPerRequest >= 5
```

PAYLOAD:

```
observed.responseSizeP95Bytes >= baseline.responseSizeP95Bytes * 3
```

E

```
observed.responseSizeP95Bytes - baseline.responseSizeP95Bytes >= 1500
```

Retorne:

```
{
```

```

"anomalyDetected": false,
"anomalyTypes": [],
"firstAnomalousSignal": null,
"gateResult": "pass",
"triggeredRules": []
}

```

Não implemente score ou níveis de severidade.

OPENAI

Reutilize a infraestrutura do Dia 1.

A IA recebe:

- baseline;
- observado;
- resultado determinístico;
- regras acionadas;
- até cinco evidências sanitizadas dos logs.

A IA pode produzir somente:

- summary;
- evidence;
- interpretation;
- probableCause;
- recommendedActions;
- limitations.

A IA não pode definir:

- anomalyDetected;
- anomalyTypes;
- firstAnomalousSignal;
- gateResult;
- triggeredRules.

Sem OPENAI_API_KEY, use fallback determinístico.

PAINEL HTML

Gere:

```

reports/day-2/anomaly-report.json
reports/day-2/anomaly-report.md
reports/day-2/anomaly-report.html

```

O HTML deve apresentar:

1. banner NORMAL ou ANOMALIA DETECTADA;
2. quatro cartões:
 - latência;
 - taxa de erro;
 - logs por requisição;
 - tamanho da resposta;
3. em cada cartão:
 - baseline;
 - observado;
 - diferença;
 - regra;
 - status textual;
4. faixa visual das 30 requisições;
5. código HTTP e duração de cada amostra em tooltip ou tabela;
6. cinco evidências dos logs;
7. explicação da IA ou fallback;
8. causa provável;
9. ações;
10. limitações.

Use apenas HTML, CSS e JavaScript inline.
Não adicione biblioteca de gráficos.

COMANDOS

Adicione:

```
npm run day2:baseline
npm run day2:check
npm run test:day2
```

day2:check deve observar a aplicação atual sem ativar cenário.

TESTES

Cubra:

- p95;
- errorRate;
- logLinesPerRequest;
- responseSizeP95Bytes;
- cenário normal;
- uma única requisição de 50 ms não gera anomalia;
- aumento relativo grande e absoluto pequeno não gera anomalia;
- cada uma das quatro regras;
- IA não altera a decisão;
- fallback sem chave;
- encerramento do processo;
- escape de HTML;
- ausência de secrets.

NÃO IMPLEMENTAR

Não adicione:

- LangSmith;
- Discord;
- OpenTelemetry;
- Grafana;
- dashboards externos;
- estatística avançada;
- machine learning;
- workflow GitHub específico;
- alertas;
- banco de dados.

Execute:

```
npm run test:day2
npm run lint
npm run test
npm run build
```

Não faça commit, push ou Pull Request.

Ao final:

- explique cada arquivo;
- mostre o fluxo;
- liste os testes;
- declare o que não foi testado externamente.

3.3 Inspeção antes da primeira execução

Antes de executar o painel, confira o que foi criado. O objetivo é evitar que a automação seja tratada como uma caixa-preta.

```
git status --short
find automation/src/day2 -maxdepth 1 -type f -print
find automation/tests -maxdepth 1 -type f -name 'day-2-*' -print
```

Abra os arquivos na ordem do fluxo. Leia primeiro observe.mjs, depois detect.mjs, explain.mjs, report.mjs e, por último, run.mjs. Ao ler, identifique a entrada, a saída e a responsabilidade de cada módulo.

```
sed -n '1,280p' automation/src/day2/observe.mjs
sed -n '1,260p' automation/src/day2/detect.mjs
sed -n '1,260p' automation/src/day2/explain.mjs
sed -n '1,300p' automation/src/day2/report.mjs
sed -n '1,320p' automation/src/day2/run.mjs
```

3.4 Validação da infraestrutura

```
npm run test:day2
npm run lint
npm run test
npm run build
```

Todos os comandos devem terminar com sucesso. Caso algum teste falhe, não avance para os cenários. Corrija primeiro a infraestrutura comum, pois todos os quatro cenários dependem dela.

3.5 Criação da baseline

```
npm run day2:baseline
cat reports/day-2/baseline.json | jq
```

A baseline representa uma execução normal naquele ambiente. Ela deve registrar 30 amostras medidas após o aquecimento. Os valores não precisam ser iguais aos exemplos do prompt. Uma máquina pode produzir **p95** de **5 ms** e outra de **20 ms**. O importante é que a baseline seja criada sem um modo de anomalia ativo.

Campo	Como interpretar
sampleCount	Quantidade de requisições utilizadas na janela.
latencyP95Ms	Valor abaixo do qual ficaram aproximadamente 95% das durações.
errorRate	Proporção de requisições com resposta de erro.
logLinesPerRequest	Média de linhas estruturadas associadas a cada requisição.
responseSizeP95Bytes	Tamanho p95 do corpo das respostas.

3.6 Execução normal e análise do painel

```
npm run day2:check
xdg-open reports/day-2/anomaly-report.html
```

Na execução normal, os quatro cartões devem permanecer com status **NORMAL**. O banner principal deve indicar que nenhuma anomalia foi detectada, e `gateResult` deve ser `pass`. Uma pequena variação de latência não deve ser suficiente para produzir uma anomalia.

```
jq '{
  anomalyDetected,
  anomalyTypes,
  firstAnomalousSignal,
  gateResult,
  triggeredRules
}' reports/day-2/anomaly-report.json
```

Avaliação da execução normal

Confirme que o painel não reage a pequenas oscilações. Uma execução normal que gera alerta indica thresholds inadequados, erro de baseline ou interferência externa. O detector deve ser capaz de reconhecer tanto o comportamento anômalo quanto o comportamento saudável.

3.7 Checkpoint da infraestrutura comum

Depois de validar a baseline, a execução normal e os testes, registre um checkpoint. Essa tag será a base do desafio final, porque contém o painel e o detector, mas ainda não contém os quatro cenários prontos.

```
git add -A
git commit -m "feat(day2): adiciona painel visual e detector"
git tag day-2-visual-base
git push -u origin feature/day-2-visual-anomaly-lab
git push origin day-2-visual-base
```

4. Cenário 1: latência intermitente

O primeiro cenário é o mais importante. A rota continuará retornando HTTP 200, mas uma em cada três requisições terá um atraso aproximado de 500 ms. O objetivo é mostrar que disponibilidade e desempenho são dimensões diferentes.

4.1 Mudança que será introduzida

```
if (mode === "latency" && requestNumber % 3 === 0) {
  await delay(500);
}
```

A condição utiliza um contador para tornar o comportamento previsível. Duas requisições permanecem rápidas e a terceira recebe o atraso. Essa periodicidade faz a p95 subir sem alterar o contrato funcional da rota.

4.2 Prompt do cenário

Adicione ao laboratório do Dia 2 somente o cenário de latência intermitente.

Crie:

```
backend/src/demo-anomalies.js
backend/test/demo-anomalies.test.js
```

Integre esse módulo da forma mais simples possível somente em GET /api/report.

Quando:

DEMO_ANOMALY_MODE=latency

a cada terceira requisição à rota deve aguardar 500 ms antes de continuar.

Requisitos:

- continuar retornando HTTP 200;
- não alterar /api/health;
- não alterar o relatório de negócio;
- não alterar o detector;
- não alterar a baseline;
- registrar um evento estruturado: demo.anomaly.latency;
- não criar atraso sem a variável.

Adicione:

```
npm run day2:scenario -- latency
```

O comando deve:

1. usar reports/day-2/baseline.json;
2. iniciar o backend com DEMO_ANOMALY_MODE=latency;
3. observar 30 requisições;
4. detectar a anomalia;
5. pedir explicação à IA;
6. gerar o painel HTML;
7. encerrar o backend.

Teste:

- duas em cada três requisições rápidas;
- uma em cada três com atraso;
- HTTP 200;
- firstAnomalousSignal=latencyP95Ms.

Não implemente outros cenários.
Não faça commit ou push.

4.3 Conferência da mudança

```
git diff day-2-visual-base -- backend/src/demo-anomalies.js backend/src/routes/report.js
backend/test/demo-anomalies.test.js
```

Antes de executar, responda: qual sinal deve mudar, qual sinal deve permanecer normal e por que o health check não deve detectar esse problema?

4.4 Execução

```
npm run test:day2
npm run test:backend
npm run day2:scenario -- latency
xdg-open reports/day-2/anomaly-report.html
```

4.5 Avaliação e análise dos resultados

O cartão de latência deve ser o principal cartão anômalo. A taxa de erro deve permanecer próxima de zero, o volume de logs deve permanecer próximo da baseline e o tamanho da resposta não deve apresentar crescimento relevante.

Elemento	Resultado esperado	Interpretação
HTTP	200 nas 30 requisições	A operação continua funcional.
latencyP95Ms	Muito acima da baseline	Uma parcela relevante das respostas ficou lenta.
errorRate	Próxima de 0	A degradação não é uma indisponibilidade.
Faixa de requisições	Marcadores lentos em periodicidade	O desvio é intermitente e previsível.
IA	Explica aumento de latência e limitações	A IA interpreta fatos já calculados.

Separe a análise em cinco níveis. Fato: a p95 aumentou. Regra: o aumento relativo e o aumento absoluto foram satisfeitos. Interpretação: existe degradação de desempenho. Hipótese: algum caminho de sucesso passou a esperar ou processar mais. Causa confirmada: o atraso periódico introduzido no código.

```
jq '{
  anomalyDetected,
  anomalyTypes,
  firstAnomalousSignal,
  gateResult,
  triggeredRules,
  baseline,
  observed,
```

```
explanation
} reports/day-2/anomaly-report.json
```

4.6 Checkpoint do cenário de latência

```
git add -A
git commit -m "feat(day2): adiciona cenário de latência"
git tag day-2-scenario-latency
git push origin feature/day-2-visual-anomaly-lab
git push origin day-2-scenario-latency
```

5. Cenário 2: erro intermitente

Neste cenário, a aplicação permanece viva, mas uma em cada cinco chamadas de relatório falha. O health check continuará respondendo normalmente. A anomalia será identificada pela taxa de erro da operação funcional.

5.1 Mudança que será introduzida

```
if (mode === "error-rate" && requestNumber % 5 === 0) {
  throw new DemoAnomalyError(
    "falha intermitente injetada para demonstração",
  );
}
```

Em 30 requisições, a expectativa é observar aproximadamente seis falhas, equivalentes a 20%. As demais chamadas continuam funcionando. Esse padrão demonstra por que uma única chamada manual pode não revelar uma degradação intermitente.

5.2 Prompt do cenário

Adicione somente o cenário de erro intermitente ao módulo backend/src/demo-anomalies.js.

Quando:

DEMO_ANOMALY_MODE=error-rate

a cada quinta requisição a GET /api/report deve lançar um erro controlado.

Requisitos:

- utilizar o error handler existente;
- aproximadamente 20% das chamadas retornam HTTP 500;
- as demais continuam HTTP 200;
- /api/health continua HTTP 200;
- registrar:
 - demo.anomaly.error-rate;
- não alterar detector, baseline ou relatório;
- nenhum comportamento muda sem a variável.

O comando:

```
npm run day2:scenario -- error-rate
```

deve gerar o painel visual e detectar:

anomalyTypes contendo error-rate;
firstAnomalousSignal igual a errorRate.

Adicione testes.

Não implemente outros cenários.
Não faça commit ou push.

5.3 Conferência da mudança

```
git diff day-2-scenario-latency -- backend/src/demo-anomalies.js backend/src/routes/report.js
backend/test/demo-anomalies.test.js
```

Observe que o detector não deve ser modificado. O cenário apenas altera o comportamento da aplicação. A regra de taxa de erro já existia desde a construção da infraestrutura comum.

5.4 Execução

```
npm run test:day2
npm run test:backend
npm run day2:scenario -- error-rate
xdg-open reports/day-2/anomaly-report.html
```

5.5 Avaliação e análise dos resultados

O cartão de taxa de erro deve indicar anomalia. A faixa das 30 requisições deverá mostrar respostas 500 em uma periodicidade aproximada de uma falha a cada cinco chamadas. O health check não aparece como causa porque o processo permanece vivo.

Elemento	Resultado esperado	Interpretação
errorRate	Aproximadamente 0,20	A operação funcional apresenta falhas intermitentes.
HTTP 500	Cerca de 6 ocorrências em 30	A falha não ocorre em todas as chamadas.
Health check	200	Disponibilidade do processo não garante confiabilidade da função.
latencyP95Ms	Pode variar, mas não é o sinal principal	A prioridade de triagem deve apontar errorRate.
IA	Relaciona taxa de erro e eventos failed	A explicação deve evitar afirmar causa sem evidência de código.

Fato: aproximadamente 20% das requisições falharam. Regra: a taxa superou 15% e aumentou pelo menos dez pontos percentuais em relação à baseline. Interpretação: há degradação intermitente da operação. Hipótese: uma condição periódica ou dependência instável está interrompendo parte das execuções. Causa confirmada: a exceção controlada inserida no código.

5.6 Checkpoint do cenário de erro

```
git add -A
git commit -m "feat(day2): adiciona cenário de erro intermitente"
```

```
git tag day-2-scenario-error-rate  
git push origin feature/day-2-visual-anomaly-lab  
git push origin day-2-scenario-error-rate
```

6. Cenário 3: excesso de logs

O terceiro cenário mostra uma anomalia operacional que pode não ser percebida pelo usuário. As respostas continuam rápidas e corretas, mas a aplicação passa a produzir muitas linhas adicionais de log. Isso aumenta volume, custo, ruído e dificuldade de investigação.

6.1 Mudança que será introduzida

```
if (mode === "noisy-logs" && requestNumber % 3 === 0) {
  for (let index = 0; index < 10; index += 1) {
    log.emit({
      level: "info",
      eventType: "demo.anomaly.noisy-log",
      phase: "functional",
      message: "linha adicional de demonstração",
    });
  }
}
```

A emissão ocorre a cada terceira requisição. O comportamento é controlado para que o aluno consiga relacionar a periodicidade do código com o crescimento da métrica `logLinesPerRequest`.

6.2 Prompt do cenário 3

Adicione somente o cenário de excesso de logs.

Quando:

DEMO_ANOMALY_MODE=noisy-logs

a cada terceira requisição a GET /api/report deve emitir 10 eventos estruturados adicionais:

eventType: demo.anomaly.noisy-log

Requisitos:

- HTTP 200;
- health check inalterado;
- sem atraso intencional;
- sem loop infinito;
- exatamente 10 eventos extras por ativação;
- não alterar detector ou baseline;
- nenhum comportamento muda sem a variável.

O comando:

`npm run day2:scenario -- noisy-logs`

deve detectar:

anomalyTypes contendo log-volume;

firstAnomalousSignal igual a logLinesPerRequest.

Adicione testes.

Não implemente outro cenário.

Não faça commit ou push.

6.3 Conferência da mudança

```
git diff day-2-scenario-error-rate -- backend/src/demo-anomalies.js backend/src/routes/report.js
backend/test/demo-anomalies.test.js
```

Verifique se os eventos extras são estruturados, limitados e emitidos somente no modo noisy-logs. Um cenário didático não pode criar um loop infinito ou preencher o disco do computador.

6.4 Execução

```
npm run test:day2
npm run test:backend
npm run day2:scenario -- noisy-logs
xdg-open reports/day-2/anomaly-report.html
```

6.5 Avaliação e análise dos resultados

O cartão de logs por requisição deve indicar anomalia. Os cartões de latência, taxa de erro e tamanho da resposta devem permanecer normais ou próximos da baseline. A análise da IA deve destacar que o desvio é operacional, não necessariamente funcional.

Elemento	Resultado esperado	Interpretação
logLinesPerRequest	Mais de três vezes a baseline e aumento superior a 5	O volume de logs cresceu de forma relevante.
HTTP	200	O usuário ainda recebe a resposta.
errorRate	Próxima de 0	Não há evidência de falha funcional.
Eventos	demo.anomaly.noisy-log repetido	As evidências apontam a origem do volume.
IA	Explica custo e ruído operacional	A causa no código só é confirmada após inspeção.

Fato: a média de linhas por requisição aumentou. Regra: o valor observado ultrapassou três vezes a baseline e cresceu mais de cinco linhas por requisição. Interpretação: há ruído operacional anormal. Hipótese: algum caminho está registrando eventos repetidos. Causa confirmada: o laço de dez emissões periódicas.

6.6 Checkpoint do cenário de logs

```
git add -A
git commit -m "feat(day2): adiciona cenário de excesso de logs"
git tag day-2-scenario-noisy-logs
git push origin feature/day-2-visual-anomaly-lab
git push origin day-2-scenario-noisy-logs
```

7. Cenário 4: payload inflado

O último cenário mantém o status HTTP 200 e o formato geral da resposta, mas multiplica o conteúdo de uma lista. O usuário pode continuar vendo dados válidos, embora a API passe a transferir uma quantidade de bytes muito maior.

7.1 Mudança que será introduzida

```
if (mode === "payload-bloat") {
  return {
    ...report,
    missingStickers: Array.from(
      { length: 8 },
      () => report.missingStickers,
    ).flat(),
  };
}
```

O cenário não altera o armazenamento. Ele modifica apenas a resposta da execução atual. Essa restrição evita efeitos colaterais permanentes e mantém a prática reproduzível.

7.2 Prompt do cenário

Adicione somente o cenário de payload inflado.

Quando:

DEMO_ANOMALY_MODE=payload-bloat

a resposta de GET /api/report deve manter seu formato geral, mas uma das listas do relatório deve ser repetida até que o corpo fique pelo menos três vezes maior que o normal.

Requisitos:

- HTTP 200;
- health check inalterado;
- não alterar o store;
- não persistir dados;
- registrar:
 - demo.anomaly.payload-bloat;
- não alterar detector ou baseline;
- nenhum comportamento muda sem a variável.

O comando:

npm run day2:scenario -- payload-bloat

deve detectar:

anomalyTypes contendo payload-bloat;
firstAnomalousSignal igual a responseSizeP95Bytes.

Adicione testes.

Não faça commit ou push.

7.3 Conferência da mudança

```
git diff day-2-scenario-noisy-logs -- backend/src/demo-anomalies.js backend/src/routes/report.js
backend/test/demo-anomalies.test.js
```

Confirme que o cenário não modifica o store e não persiste dados. A anomalia deve existir apenas na resposta produzida quando o modo payload-bloat estiver ativo.

7.4 Execução

```
npm run test:day2
npm run test:backend
npm run day2:scenario -- payload-bloat
xdg-open reports/day-2/anomaly-report.html
```

7.5 Avaliação e análise dos resultados

O cartão de tamanho da resposta deve ficar anômalo. A taxa de erro deve permanecer em zero, e o status HTTP deve continuar 200. A latência pode crescer um pouco devido ao aumento do corpo, mas o primeiro sinal anômalo esperado é responseSizeP95Bytes.

Elemento	Resultado esperado	Interpretação
responseSizeP95Bytes	Mais de três vezes a baseline e aumento superior a 1.500 bytes	A resposta cresceu de forma operacionalmente relevante.
HTTP	200	O contrato de transporte aparenta sucesso.
Formato geral	Preservado	A anomalia pode passar despercebida em um teste superficial.
Persistência	Sem alteração	O cenário é temporário e reproduzível.
IA	Explica consumo de rede e payload	A hipótese deve ser conferida no código.

Fato: o tamanho p95 da resposta aumentou. Regra: o valor ficou pelo menos três vezes maior e cresceu mais de 1.500 bytes. Interpretação: a API passou a transferir conteúdo excessivo. Hipótese: uma coleção foi duplicada ou expandida. Causa confirmada: a repetição da lista no retorno do relatório.

7.6 Checkpoint do cenário de payload

```
git add -A
git commit -m "feat(day2): adiciona cenário de payload inflado"
git tag day-2-scenario-payload-bloat
git push origin feature/day-2-visual-anomaly-lab
git push origin day-2-scenario-payload-bloat
```

8. Comparação dos quatro cenários

Depois de executar os quatro cenários, compare os resultados. Cada cenário altera uma dimensão diferente, embora todos utilizem a mesma rota, o mesmo observador, a mesma baseline e o mesmo painel.

Cenário	HTTP	Sinal principal	Comportamento visual	Ideia central
Latência	200	latencyP95Ms	Marcadores lentos e cartão de latência anômalo	Disponibilidade não garante desempenho.
Erro intermitente	200 e 500	errorRate	Falhas periódicas na faixa de requisições	Health check não garante confiabilidade funcional.
Excesso de logs	200	logLinesPerRequest	Cartão de volume operacional anômalo	Uma anomalia pode afetar custo e investigação.
Payload inflado	200	responseSizeP95Bytes	Cartão de tamanho anômalo	Resposta válida também pode ser ineficiente.

8.1 Como interpretar a análise da IA

A explicação deve ser lida em camadas. Não confunda uma hipótese plausível com uma causa confirmada. Utilize a estrutura abaixo em todas as avaliações.

Camada	Pergunta	Exemplo de latência
Fato	O que foi medido?	A p95 passou de 8 ms para 506 ms.
Regra	Qual condição foi satisfeita?	Aumento relativo de 3 vezes e absoluto de 150 ms.
Interpretação	O que o desvio representa?	Existe degradação intermitente de desempenho.
Hipótese	Que explicação deve ser investigada?	Uma espera ou processamento adicional foi introduzido.
Causa confirmada	O que o código comprovou?	Uma em cada três requisições recebe delay de 500 ms.

Regra de confiança

A IA não vê automaticamente toda a causa no código. Ela interpreta os dados enviados. A causa só deve ser tratada como confirmada depois da inspeção dos logs, do diff e da implementação.

8.2 Confirmação após a correção

Uma correção não termina quando o código parece certo. Ela termina quando uma nova execução mostra que o sinal voltou à faixa normal. Após remover ou corrigir o comportamento anômalo, execute o check sem modo de demonstração.

```
npm run day2:check
xdg-open reports/day-2/anomaly-report.html
```

O banner deve voltar para NORMAL, anomalyDetected deve ser false e gateResult deve ser pass. Registre a comparação antes e depois para demonstrar que a normalização foi medida, e não apenas presumida.

9. Desafio final em grupos

O desafio será realizado em duas rodadas. Em cada rodada, um grupo introduz uma anomalia controlada e o outro grupo investiga, explica, corrige e mede novamente. O grupo investigador não será informado sobre o tipo de anomalia.

9.1 Preparação comum

O desafio começa a partir da tag day-2-visual-base. Essa tag contém o observador, o detector, a IA e o painel, mas não contém os cenários prontos. Cada grupo deverá gerar uma baseline normal na própria máquina antes de receber a branch com a anomalia.

```
git fetch --all --tags
git switch --detach day-2-visual-base
npm ci
npm run day2:baseline
cp reports/day-2/baseline.json /tmp/day2-baseline.json
```

A cópia em /tmp protege a baseline de alterações acidentais durante a troca de branches. O grupo não pode editar esse arquivo para facilitar a detecção.

9.2 Cartas de anomalia

Carta	Mudança permitida	Sinal esperado
A	Atraso periódico entre 400 e 700 ms, mantendo HTTP 200.	latencyP95Ms
B	Falha controlada em aproximadamente uma de cada cinco chamadas.	errorRate
C	Emissão periódica de oito a doze logs estruturados extras.	logLinesPerRequest
D	Aumento do corpo da resposta para pelo menos três vezes o normal.	responseSizeP95Bytes

9.3 Regras para o grupo que injeta

O grupo responsável pela injeção sorteia uma carta e cria uma branch a partir de day-2-visual-base. A alteração deve permanecer pequena e controlada. O objetivo não é derrubar toda a aplicação, e sim produzir um desvio que o painel consiga evidenciar.

- não alterar observe.mjs, detect.mjs, explain.mjs, report.mjs ou run.mjs;
- não alterar a baseline, os thresholds ou o arquivo de relatório;
- não quebrar /api/health;
- não adicionar credenciais ao repositório;
- não informar ao outro grupo qual carta foi utilizada;
- executar testes antes de enviar a branch.

9.4 Prompts curtos para a injeção

O grupo pode utilizar o Claude Code para acelerar a alteração. O prompt deve ser simples e não pode modificar o detector. Escolha apenas um dos modelos abaixo de acordo com a carta sorteada.

Carta A: latência

Introduza uma anomalia controlada de latência na branch atual.

Em GET /api/report, faça uma em cada três requisições aguardar entre 400 e 700 ms. A rota deve continuar retornando HTTP 200 e /api/health deve permanecer inalterado.

Registre um evento estruturado para a ocorrência.

Não altere o detector, a baseline, o painel, os thresholds ou os scripts do laboratório. Não crie variável de ambiente para ativar o comportamento. A anomalia deve existir diretamente nesta branch do desafio.

Adicione testes. Não faça commit nem push.

Carta B: erro intermitente

Introduza uma anomalia controlada de erro intermitente na branch atual.

Em GET /api/report, faça aproximadamente uma em cada cinco requisições lançar um erro controlado e passar pelo error handler existente. As outras chamadas devem continuar retornando HTTP 200 e /api/health deve permanecer inalterado.

Registre um evento estruturado para a ocorrência.

Não altere o detector, a baseline, o painel, os thresholds ou os scripts do laboratório. Não crie variável de ambiente para ativar o comportamento. A anomalia deve existir diretamente nesta branch do desafio.

Adicione testes. Não faça commit nem push.

Carta C: excesso de logs

Introduza uma anomalia controlada de excesso de logs na branch atual.

Em GET /api/report, faça uma em cada três requisições emitir entre oito e doze eventos estruturados adicionais. A resposta deve continuar HTTP 200, sem atraso intencional, e /api/health deve permanecer inalterado.

Não altere o detector, a baseline, o painel, os thresholds ou os scripts do laboratório. Não crie variável de ambiente para ativar o comportamento. A anomalia deve existir diretamente nesta branch do desafio.

Adicione testes. Não faça commit nem push.

Carta D: payload inflado

Introduza uma anomalia controlada de payload inflado na branch atual.

Em GET /api/report, preserve o formato geral da resposta, mas aumente uma lista até que o corpo fique pelo menos três vezes maior. Não altere o store, não persista dados e mantenha HTTP 200 e /api/health inalterado.

Não altere o detector, a baseline, o painel, os thresholds ou os scripts do laboratório. Não crie variável de ambiente para ativar o comportamento. A anomalia deve existir diretamente nesta branch do desafio.

Adicione testes. Não faça commit nem push.

9.5 Rodada 1

Na primeira rodada, o Grupo A injeta e o Grupo B investiga.

```
# Grupo A
git switch --detach day-2-visual-base
git switch -c challenge/round-1-anomaly-a

# Depois de aplicar uma carta e validar
git add -A
git commit -m "challenge: injeta anomalia da rodada 1"
git push -u origin challenge/round-1-anomaly-a
```

O Grupo B deve preparar a baseline limpa antes de trocar para a branch do Grupo A. Depois, cria sua própria branch de correção.

```
# Grupo B
git switch --detach day-2-visual-base
npm run day2:baseline
cp reports/day-2/baseline.json /tmp/day2-baseline.json

git fetch origin
git switch --track origin/challenge/round-1-anomaly-a
git switch -c challenge/round-1-fix-b
```

9.6 Processo obrigatório de investigação

Nos primeiros cinco minutos, o grupo investigador não pode utilizar git diff. A intenção é formular uma hipótese a partir do comportamento antes de procurar diretamente a alteração.

1. Executar o check utilizando a baseline protegida.
2. Abrir o painel HTML e identificar o cartão anômalo.
3. Comparar baseline, observado e regra acionada.
4. Ler a faixa das 30 requisições e as evidências dos logs.
5. Ler a análise da IA e separar fato, interpretação e hipótese.
6. Formular uma hipótese própria antes de abrir o código.
7. Inspeccionar os logs e, somente depois, o código e o diff.
8. Corrigir a causa confirmada sem modificar o detector.
9. Executar novamente e comprovar a normalização.
10. Registrar o commit de correção e enviar a branch.

```
npm run day2:check -- --baseline /tmp/day2-baseline.json
xdg-open reports/day-2/anomaly-report.html
```

Depois de corrigir:

```
npm run test:day2
npm run test:backend
npm run day2:check -- --baseline /tmp/day2-baseline.json

# O painel deve indicar NORMAL

git add -A
git commit -m "fix: normaliza comportamento da rodada 1"
git push -u origin challenge/round-1-fix-b
```

9.7 Rodada 2

Na segunda rodada, os papéis são invertidos. O Grupo B injeta uma carta diferente e o Grupo A investiga.

```
# Grupo B
git switch --detach day-2-visual-base
git switch -c challenge/round-2-anomaly-b

# Aplicar uma carta diferente da rodada 1
git add -A
git commit -m "challenge: injeta anomalia da rodada 2"
git push -u origin challenge/round-2-anomaly-b

# Grupo A
git switch --detach day-2-visual-base
npm run day2:baseline
cp reports/day-2/baseline.json /tmp/day2-baseline.json

git fetch origin
git switch --track origin/challenge/round-2-anomaly-b
git switch -c challenge/round-2-fix-a
```

O Grupo A repete o processo obrigatório de investigação. Ao final, registra a correção.

```
git add -A
git commit -m "fix: normaliza comportamento da rodada 2"
git push -u origin challenge/round-2-fix-a
```

9.8 Critério de conclusão

A rodada não termina quando o grupo encontra o trecho alterado. Ela termina quando o painel produzido após a correção apresenta os resultados abaixo.

```
{
  "anomalyDetected": false,
  "anomalyTypes": [],
  "firstAnomalousSignal": null,
  "gateResult": "pass"
}
```

9.9 Entrega dos grupos

Cada grupo deverá apresentar uma síntese curta e baseada em evidências. A apresentação deve seguir a sequência abaixo.

- cartão e sinal que ficaram anômalos;
- valor da baseline e valor observado;
- regra que foi acionada;
- evidências dos logs e da faixa de requisições;
- resumo produzido pela IA;
- hipótese formulada antes da inspeção do código;
- causa confirmada no código;
- correção realizada;
- resultado do painel após a correção.

10. Como adaptar o laboratório para outra aplicação

A tecnologia pode mudar, mas o processo permanece. Para aplicar a mesma abordagem em outro sistema, escolha uma operação relevante, defina poucos sinais, meça o comportamento normal e construa regras que possam ser explicadas.

Etapa	Pergunta orientadora	Exemplo
1. Escolher a operação	Qual fluxo é importante para o usuário ou negócio?	Checkout, upload, busca, geração de relatório.
2. Escolher sinais	Que comportamento pode se degradar?	Latência, erros, logs, payload, tokens.
3. Coletar amostras	Como cada execução será medida?	Status, duração, bytes, eventos.
4. Criar baseline	Como o sistema se comporta em condição normal?	Janela de 30 requisições.
5. Escrever regras	Qual diferença é relevante?	Limite relativo combinado com absoluto.
6. Reunir evidências	O que sustenta a decisão?	Métrica, requestId, logs e códigos HTTP.
7. Explicar com IA	Como transformar fatos em uma análise compreensível?	Resumo, hipótese, ações e limitações.
8. Medir novamente	A correção devolveu o sistema à faixa normal?	Novo painel com <code>anomalyDetected=false</code> .

Evite começar por ferramentas complexas. Primeiro domine a relação entre sinal, baseline, regra, evidência e ação. Depois, plataformas de observabilidade podem automatizar e escalar o mesmo raciocínio.

11. Síntese final

Uma anomalia não é apenas um erro. Ela é um desvio mensurável em relação a um comportamento de referência. Neste laboratório, o detector comprova o desvio por regras simples, o painel torna o comportamento visual, os logs fornecem evidências e a IA transforma essas evidências em uma explicação compreensível.

Ideia central

Primeiro medimos e detectamos. Depois utilizamos a IA para explicar. A correção só é concluída quando uma nova medição confirma que o comportamento voltou ao normal.